

Advanced Programming

Object Oriented Programming 2 (OOP)



What have we learned already?

- What a **class** is
- What an **object** is
- How **state and behavior** belong together
- How **encapsulation protects internal structure**
- How Python integrates objects via **magic methods**
- The difference between **object identity** (**is**) and **equality** (**==**)
- Why a class should have **one clear responsibility (SRP)**

Today's Topics

Code Reuse

- Inheritance (**is-a** relationship)
- Using `super()`

Flexible Behavior

- Polymorphism
- Method Overriding
- Duck Typing

Object Design

- Liskov Substitution Principle (LSP)

Advanced Class Mechanics

- Multiple Inheritance
- Method Resolution Order (MRO)
- Static Methods (different from Class Methods)

Interfaces

- Abstract Base Classes (ABC)



A Design Problem

```
class Lecturer:
    def __init__(self, firstname, lastname):
        self.firstname = firstname
        self.lastname = lastname

    @property
    def firstname(self):
        return self._firstname

    @firstname.setter
    def firstname(self, value):
        if not value:
            raise ValueError("...")
        self._firstname = str(value)

    ...
```

```
class Student:
    def __init__(self, firstname, lastname):
        self.firstname = firstname
        self.lastname = lastname

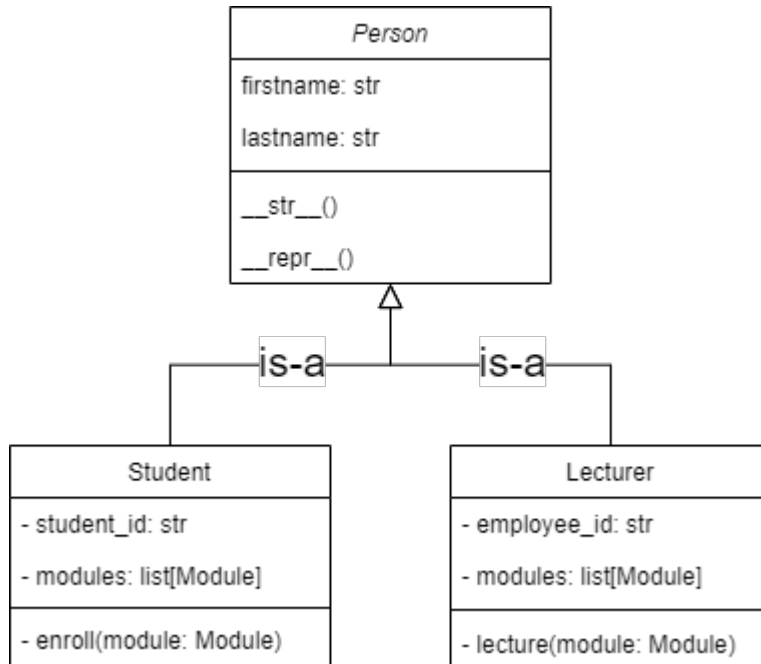
    @property
    def firstname(self):
        return self._firstname

    @firstname.setter
    def firstname(self, value):
        if not value:
            raise ValueError("...")
        self._firstname = str(value)

    ...
```

What problem do you see here?

Inheritance: inherit structure and behavior



Generalization

The parent class (Person) defines **common attributes and methods** shared by the subclasses.

Specialization

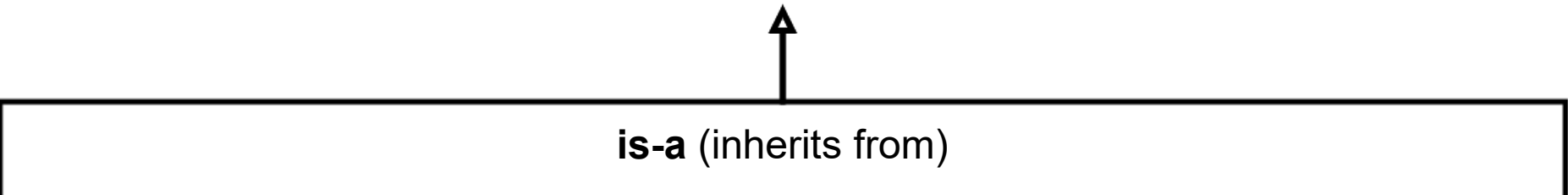
The subclasses (Student and Lecturer) specialize the parent class and **all inherit** its attributes and methods.

Inheritance models an **is-a** relationship between classes.

Inheritance: Implementation

```
class Person:  
  
    def __init__(self, firstname: str, lastname: str) -> None:  
        self.firstname = firstname  
        self.lastname = lastname
```

is-a (inherits from)



```
class Student(Person):  
    pass
```

```
class Lecturer(Person):  
    pass
```

`super()`: Reusing behavior from the parent class

`super()` allows a subclass to reuse any methods from the parent class

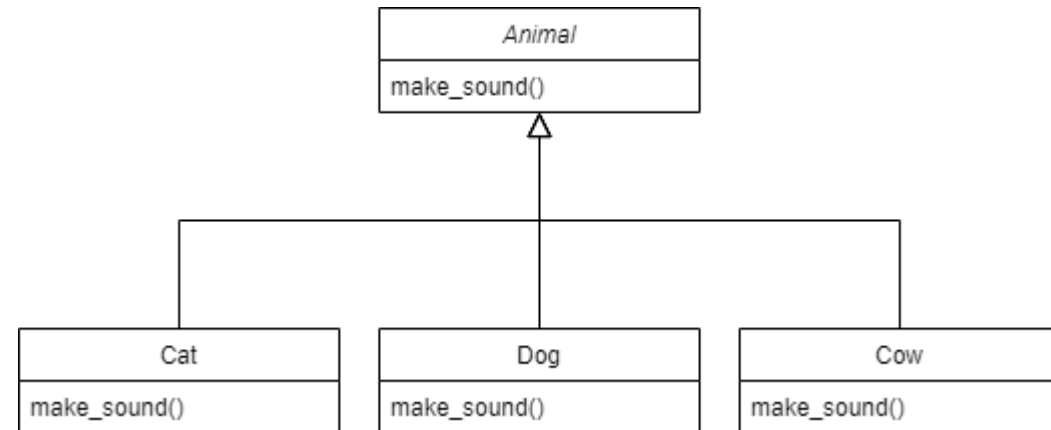
```
class Student(Person):  
  
    def __init__(self, firstname: str, lastname: str, student_id: str) -> None:  
        → super().__init__(firstname, lastname)  
        self.student_id: str = student_id  
        self._modules: list[Module] = []
```

- First, the initializer `__init__` of `super()` is called and initializes **inherited attributes**
- Afterwards, **subclass-specific attributes** are initialized (`student_id` and `_modules`)

Polymorphism: one method, different behaviors

Polymorphism means that the **same method** can **behave differently** depending on the object

- All animals can make a sound
- A cat **is-an** animal but it **meows**
- A dog **is-an** animal but it **barks**
- A cow **is-an** animal but it **moos**



Polymorphism means “*many forms*” – the “*shapeshifter*” of OOP

Overriding: change the behavior

A subclass overrides a method by redefining a method with the same name as in the parent class

```
class Animal:  
    def make_sound(self):  
        pass
```

```
class Cat(Animal):  
    def make_sound(self):  
        print("meow")
```

```
class Dog(Animal):  
    def make_sound(self):  
        print("woof")
```

```
class Cow(Animal):  
    def make_sound(self):  
        print("moo")
```

Overriding means **redefining** a parent method in a **subclass** to **change** inherited behavior.

Duck Typing: behavior instead of type

In Python, an object can be used if it implements the required methods, its type does not matter

```
class Cat(Animal):  
    def make_sound(self):  
        print("meow")  
  
class Dog(Animal):  
    def make_sound(self):  
        print("woof")  
  
class Robot: ←  
    def make_sound(self):  
        print("beep")
```



Not an animal

```
if __name__ == "__main__":  
    things = [Cat(), Dog(), Robot()]  
  
    for t in things:  
        t.make_sound()
```

```
meow  
woof  
beep
```

If it **looks like a duck**, **swims like a duck**, and **quacks like a duck**, then it probably is a **duck**.

Liskov Substitution Principle

A subclass must be usable **wherever the parent type is expected** without breaking the program.

- Code should use **the parent abstraction**
- Every subclass must keep the **expected behavior**
- Replacing one subtype with another should still work correctly

If a program works with a **parent class**, it should **also work** with any of its **subclasses**.

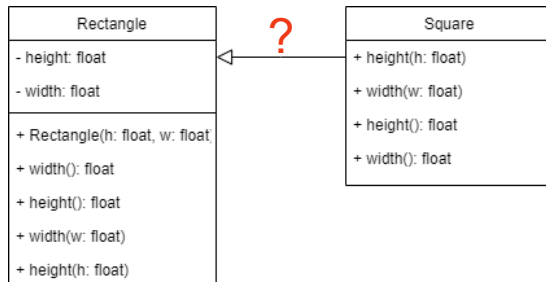


Inheritance: The trap

Inheritance looks simple – but it is not!

- Inheritance creates **strong coupling between classes**
- A subclass must respect the **behavior and structure of its parent**
- Changes in the parent class **affect all subclasses**

Mathematically every square **is-a** rectangle with special constraints: the sides must be equal.



Inheritance is about
structure and behavior!

Should square inherit from rectangle?

The answer is...

A square should usually **NOT** inherit from rectangle

Mathematically a square **is-a** rectangle – but in software their **behavior is different!**

```
class Rectangle:
    @width.setter
    def width(self, w):
        self._width = w

    @height.setter
    def height(self, h):
        self._height = h
```



```
class Square(Rectangle):
    @width.setter
    def width(self, w):
        self._width = w
        self._height = w

    @height.setter
    def height(self, h):
        self._width = h
        self._height = h
```

```
def resize(rect: Rectangle):
    rect.width = 5
    rect.height = 4
```

Expected result:

```
width = 5
height = 4
```

But with Square:

```
width = 4
height = 4
```



Substitution Principle is violated

The Solution

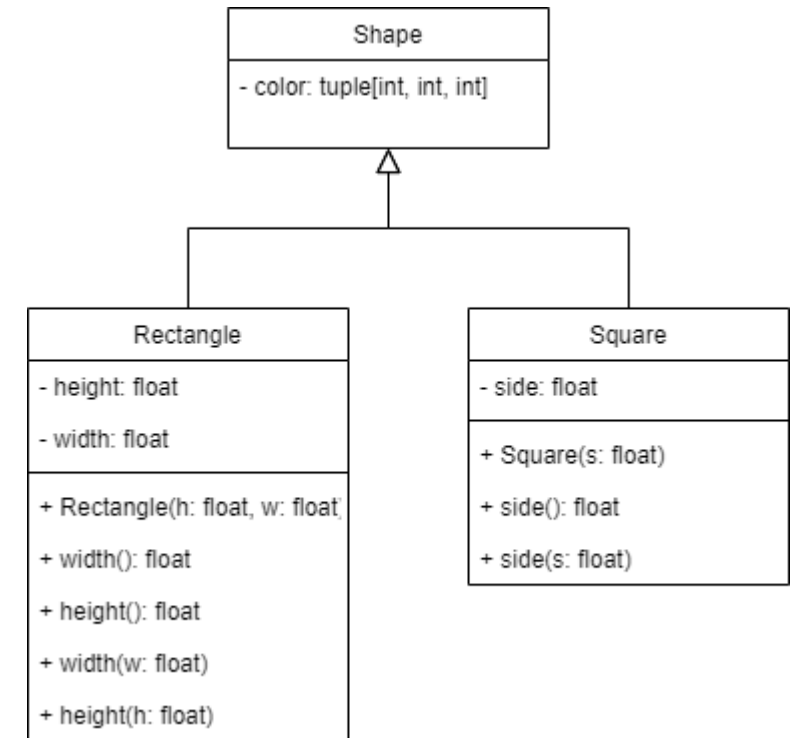
The **structure and behavior** of square is different

- Create the class independently of rectangle
- Different behavior → different abstraction.

But is this not contradictory to Polymorphism and Overriding?

- Polymorphism requires **compatible behavior**,
not **different semantics**.

 A wrong inheritance hierarchy is **difficult** to change later.



The Problem with Inheritance

The class design assumes: every bird can `fly()`

- A penguin **is-a** bird – but it cannot `fly()`
- The penguin must break the behavior



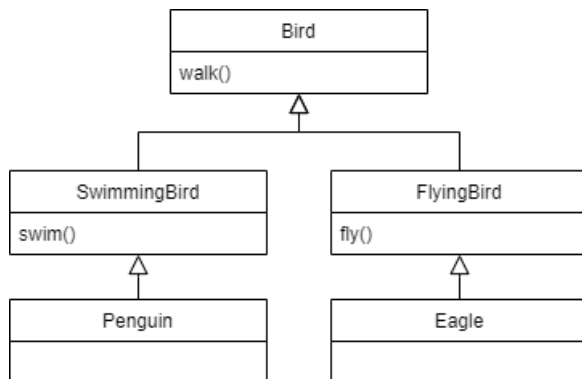
Substitution Principle is violated

```
class Bird:
    def fly(self):
        print("flying")

class Eagle(Bird):
    def fly(self):
        print("flying like a sparrow")

class Penguin(Bird):
    def fly(self):
        raise Exception("Penguins cannot fly")
```

Would this be the solution?

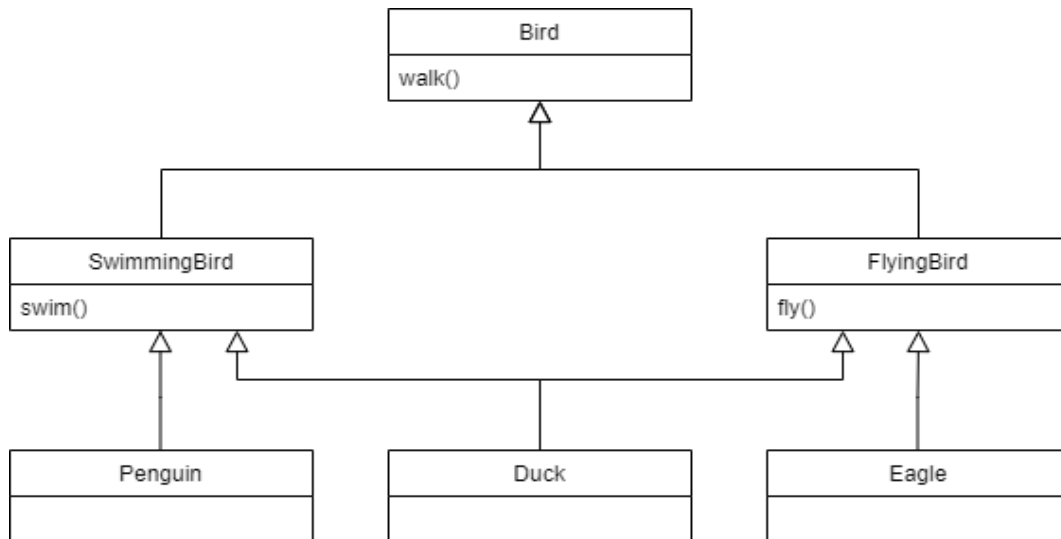


What about a Duck that can `fly()`, `swim()`, and `walk()`?



Inheritance can be complicated and a design trap

Multiple Inheritance: The Jack-of-all-trades?



Multiple inheritance can lead to
complex and fragile class hierarchies.

```
class Bird:
    def walk(self):
        print("walking")

class FlyingBird(Bird):
    def fly(self):
        print("flying")

class SwimmingBird(Bird):
    def swim(self):
        print("swimming")

class Eagle(FlyingBird):
    pass

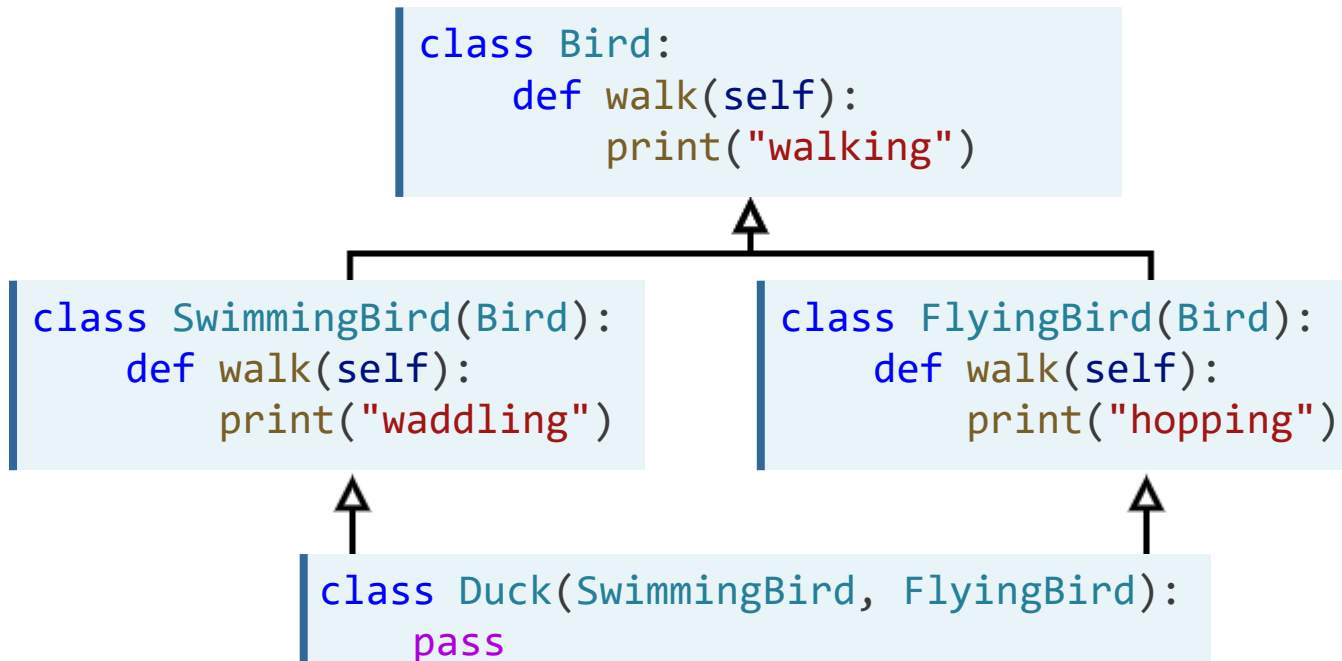
class Penguin(SwimmingBird):
    pass

class Duck(FlyingBird, SwimmingBird):
    pass
```

We will learn Design Patterns to solve this problem even more elegant

Method Resolution Order (MRO)

What if multiple parent classes define the same method?



Python searches in the order of inheritance
Duck→SwimmingBird→FlyingBird→Bird

This can be checked with `Duck.mro()`

```

if __name__ == "__main__":
    duck = Duck()
    duck.walk()
    
```

waddling

The MRO guarantees a **consistent and predictable behavior**, even with multiple inheritance.

When it gets complex

Problem

- Two parent classes implement the **same method**.
- Python's **Method Resolution Order (MRO)** automatically selects **one implementation based on the inheritance order**.
- But sometimes the subclass needs the **other implementation**.

Solution

- Override the method and explicitly call the parent implementation that should be used.

```
class Seagull(SwimmingBird, FlyingBird):  
    def walk(self):  
        # explicitly choose the implementation  
        FlyingBird.walk(self)
```



Multiple inheritance can easily become complex and error-prone.

Abstract Base Class (ABC): Formal contract for subclasses

An Abstract Base Class ([ABC](#)) defines a common interface that all subclasses must follow.

- Defines a **common interface**
- Can provide **shared behavior**
- Subclasses must implement abstract methods
- You **cannot instantiate an abstract class** directly

To create an **Abstract Base Class**

- Inherit from [ABC](#) found in the [abc](#) package
- Define the methods for the interface
- Annotate the methods with `@abstractmethod`

```
from abc import ABC, abstractmethod

class TaxableItem(ABC):
    """
    ABC for items that can calculate VAT.
    """

    def __init__(self, net_price: float):
        self.net_price = net_price

    @abstractmethod
    def calculate_vat(self) -> float:
        """
        Calculate the VAT for this item.
        Must be implemented by subclasses.
        """
        pass
```

Abstract Base Class (ABC):

```
from abc import ABC, abstractmethod

class TaxableItem(ABC):

    ...

    @abstractmethod
    def calculate_vat(self) -> float:
        pass
```

```
class RegularProduct(TaxableItem):
    VAT_RATE = 0.081

    def calculate_vat(self) -> float:
        return (
            self.net_price *
            RegularProduct.VAT_RATE
        )
```

```
class ReducedProduct(TaxableItem):
    VAT_RATE = 0.026

    def calculate_vat(self) -> float:
        return (
            self.net_price *
            ReducedProduct.VAT_RATE
        )
```

```
class TaxFreeProduct(TaxableItem):

    def calculate_vat(self) -> float:
        return 0.0
```

Using the established class hierarchy

ShoppingCart does not need to know

- Details about the concrete Product
- What kind of taxes to apply
- All possible subclasses

```
if __name__ == "__main__":
    cart = ShoppingCart()

    cart.add_item(RegularProduct("Headphones", 100.00))
    cart.add_item(ReducedProduct("Bread", 10.00))
    cart.add_item(TaxFreeProduct("Postage Stamp", 5.00))

    print(f"Net total:    {cart.total_net():.2f} CHF")
    print(f"VAT total:    {cart.total_vat():.2f} CHF")
    print(f"Gross total: {cart.total_gross():.2f} CHF")
```

```
class ShoppingCart:
    def __init__(self):
        self._items: list[TaxableItem] = []

    def add_item(self, item: TaxableItem) -> None:
        self._items.append(item)

    def total_net(self) -> float:
        return sum(item.net_price for item in self._items)

    def total_vat(self) -> float:
        return sum(item.calculate_vat() for item in self._items)

    def total_gross(self) -> float:
        return sum(item.gross_price() for item in self._items)
```

This is resulting in

- ✓ clear responsibilities
- ✓ clear interfaces
- ✓ clean substitution

The 3 Rules of Good Inheritance

1. Subclasses must behave like their parent (LSP)

A subclass should work anywhere the parent type is expected without breaking program behavior.

2. Use inheritance only for true “is-a” relationships

A subclass should represent a specialized form of the parent class.

3. Prefer composition when behavior differs

If classes share functionality but not behavior, composition is usually the better design.

(More about this later)

Key Takeaways

Inheritance

Inheritance allows a class to reuse behavior from another class. However, it creates a strong structural dependency between classes.

Polymorphism

Objects of different classes can be used through the same interface. The behavior depends on the concrete object.

Duck Typing

In Python, behavior matters more than the explicit type.

Liskov Substitution Principle (LSP)

Subclasses must be usable wherever the parent type is expected without breaking program behavior.

Abstract Base Classes (ABC)

ABCs define required methods and ensure that subclasses implement a consistent interface.

Design of Inheritance

Inheritance hierarchies must be designed carefully. Incorrect hierarchies are difficult to change later.